NAME: AYYASABGARI JAVEED BASHA

USN:23BTRCL167

COURSE:CSE-AIML-A

HCL_ID:HCLTFP2269278

SECTION A: ADVANCED SQL || Maximum Marks: 20

Tasks:

a. Display employee names, department names, and manager names using joins. (2 marks)

```
62
63
64
65  select e.EmployeeName,d.DepartmentName,d.Manager_name from Employees e inner join Departments d on e.EmployeeID=d.DepartmentID
66
```

Output:

Results | Messages

	EmployeeName	DepartmentName	Manager_name
1	javeed	IT	Ramesh
2	sailaja	Operations	Praveen
3	sandeep	Research	Rahul
4	teja	Development	Vikram
5	naeeem	Support	Anil
6	basha	Testing	Sunil
7	marina	Security	Rajesh
8	jarian	Legal	Manoj
9	prabhas	Administration	Sai Kumar
10	dikshita	Logistics	Venkat
11	harsha	Analytics	Naveen

b. Find employees earning more than their department's average salary. (2 marks

```
SELECT
    e.EmployeeID,
    e.EmployeeName,
    e.DepartmentID,
    e.Salary
FROM Employees e
WHERE e.Salary > (
    SELECT AVG(e2.Salary)
    FROM Employees e2
    WHERE e2.DepartmentID = e.DepartmentID
);
```

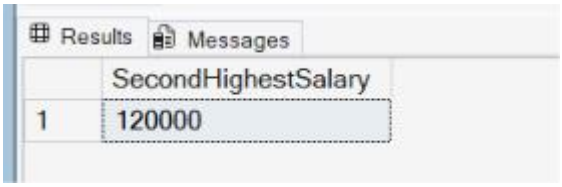
Output:



c. Find the second-highest distinct salary without using LIMIT, TOP, or FETCH FIRST. (2 marks)

```sql
SELECT MAX(Salary) AS SecondHighestSalary
FROM Employees
WHERE Salary < (
    SELECT MAX(Salary)
    FROM Employees
);
```

Output:



d. Rank employees by salary within each department and display the top three employees per department, retaining ties at the cutoff. (2 marks)

```sql
WITH RankedEmployees AS (
    SELECT
        EmployeeID,
        EmployeeName,
        DepartmentID,
        Salary,
        RANK() OVER (
            PARTITION BY DepartmentID
            ORDER BY Salary DESC
        ) AS SalaryRank
    FROM Employees
)
SELECT
    EmployeeID,
    EmployeeName,
    DepartmentID,
    Salary,
    SalaryRank
FROM RankedEmployees
WHERE SalaryRank <= 3
ORDER BY DepartmentID, Salary DESC;
```

Output:

```
116 || ORDER BY DepartmentID, Salary DESC;
```

82 % 16 0

Results Messages

	EmployeeID	EmployeeName	DepartmentID	Salary	SalaryRank
1	116	NULL	NULL	NULL	1
2	101	javeed	9631	70000	1
3	102	hussain	9632	60000	1
4	103	sasi	9633	65000	1
5	104	vasmi	9634	40000	1
6	105	pranavi	9635	67000	1
7	106	sailaja	9636	30000	1
8	107	sandeep	9637	80000	1
9	108	teja	9638	90000	1
10	109	naeeem	9639	100000	1
11	110	basha	9640	22000	1
12	111	marina	9641	78000	1
13	112	jarian	9642	49000	1
14	113	prabhas	9643	34000	1
15	114	dikshita	9644	120000	1
16	115	harsha	9645	760000	1

Question 2: Sales Analytics and Query Optimization (6 Marks)

a. **Calculate total revenue for each product. (2 marks)**

```
select productName,sum(unitprice) from sales group by productName
```

Output:

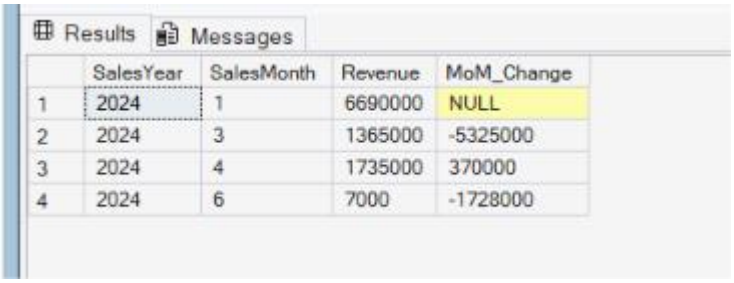
Results Messages

	productName	(No column name)
1	bottle	25000
2	charger	862000
3	gasstove	955000
4	jar	455000
5	laptop	710000
6	phone	770000
7	shoes	110000
8	slipers	355000

b. Calculate monthly revenue and month-over-month revenue change using a window function. (2 marks)

```sql
WITH MonthlyRevenue AS (
    SELECT
        YEAR(SalesDate) AS SalesYear,
        MONTH(SalesDate) AS SalesMonth,
        SUM(quantity * unitprice) AS Revenue
    FROM Sales
    GROUP BY YEAR(SalesDate), MONTH(SalesDate)
)
SELECT
    SalesYear,
    SalesMonth,
    Revenue,
    Revenue - LAG(Revenue) OVER (
        ORDER BY SalesYear, SalesMonth
    ) AS MoM_Change
FROM MonthlyRevenue
ORDER BY SalesYear, SalesMonth;
```

Output:



Results | Messages

	SalesYear	SalesMonth	Revenue	MoM_Change
1	2024	1	6690000	NULL
2	2024	3	1365000	-5325000
3	2024	4	1735000	370000
4	2024	6	7000	-1728000

c. Rewrite the following query to improve the potential for index usage and explain why your version may perform better. (2 marks)

ANS: SELECT *

FROM Employees

WHERE YEAR(JoiningDate) = 2025;

Assume JoiningDate may be indexed.

Output:

SELECT *

FROM Employees

WHERE JoiningDate >= '2025-01-01'

AND JoiningDate < '2026-01-01';

Question 3: SQL Troubleshooting (6 Marks)

A)How does an execution plan help identify expensive operations? (2 Marks)

ANS:

- It identifies expensive operations such as table scans, index scans, joins, and sorting.
- It displays the estimated or actual cost, number of rows processed, and resource usage, helping us optimize slow queries.

b) Recommend an index strategy for DepartmentID and Salary. (2 Marks)

ANS:

CREATE INDEX idx_employee_dept_salary

ON Employees (DepartmentID, Salary);

Reasoning:

- The index can improve queries filtering by both columns and queries filtering by DepartmentID alone.

c) Write a query to identify duplicate customer email addresses. (2 Marks)

ANS:

SELECT Email, COUNT(*) AS DuplicateCount

FROM Customers

GROUP BY Email

HAVING COUNT(*) > 1;

SECTION B: LINUX || Maximum Marks: 15

SECTION B: LINUX — Maximum Marks: 15

Question 4: Log Analysis (5 Marks)

a. Find all lines containing ERROR (1 mark)

ANS:grep "ERROR" /var/log/application.log

b. Display the 20 most recent error entries in the file (1 mark)

ANS:grep "ERROR" /var/log/application.log | tail -n 20

c. Count the total number of error entries (1 mark)

ANS:grep -c "ERROR" /var/log/application.log

d. Extract error entries for a specified date into a separate file (1 mark)

ANS:grep "^2026-10-28.*ERROR" /var/log/application.log > /var/log/errors-2026-10-28.log

e. Explain how you would calculate the number of errors occurring in each hour (1 mark)

ANS:grep "ERROR" /var/log/application.log | awk '{print substr($2,1,2)}' | sort | uniq -c

Explanation: The grep command filters error entries, awk extracts the hour from the timestamp, sort groups identical hours together, and uniq -c counts the errors for each hour.

Question 5: Permissions and System Troubleshooting (5 Marks)

a. Create a group named finance and a report file (1 mark)

ANS: sudo groupadd finance

sudo touch report.txt

b. Set permissions so the owner can read and write, the finance group can read, and other users have no access (1 mark)

ANS: sudo chown "$USER":finance report.txt

sudo chmod 640 report.txt

c. Identify the processes consuming the most CPU (1 mark)

ANS: ps aux --sort=-%cpu | head

d. Display memory usage and filesystem disk utilization (1 mark)

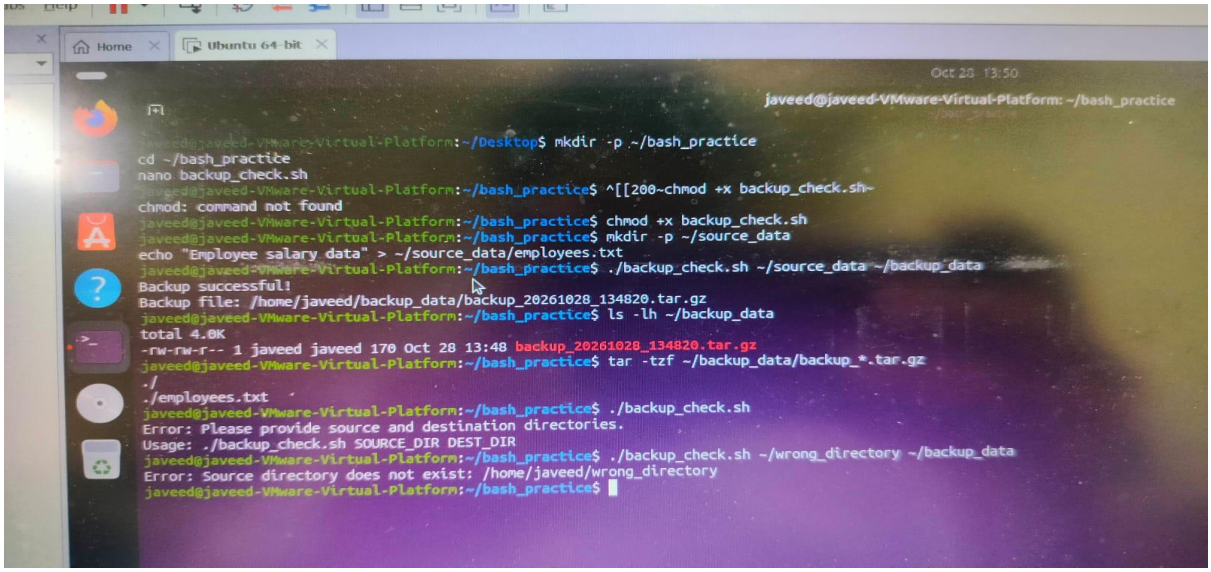
ANS: free -h

df -h

e. Find files larger than 100 MB under a specified application directory (1 mark)

A NS: find /path/to/application -type f -size +100M

Question 6: Bash Scripting (5 Marks)



SECTION C: PYSPARK DATA ENGINEERING || Maximum Marks: 20

Question 7: PySpark DataFrame Operations (6 Marks)

```
#Initialize SparkSession
from pyspark.sql import SparkSession
from pyspark.sql.functions import col

spark = SparkSession.builder.appName("EmployeeAnalytics").getOrCreate()

# Create 15 sample employee records
data = [
    (1, "Ramesh Kumar", "IT", 100000, "2021-01-10"),
    (2, "Priya Sharma", "HR", 80000, "2021-03-15"),
    (3, "Amit Patel", "Finance", 90000, "2021-05-20"),
    (4, "Sneha Reddy", "Sales", 85000, "2021-07-01"),
    (5, "Yekkaluri Anirudh", "IT", 75000, "2022-02-15"),
    (6, "Vikram Rathod", "IT", 75000, "2022-04-10"),
    (7, "Anjali", "IT", 60000, "2023-01-05"),
    (8, "Leo Das", "IT", 50000, "2023-06-18"),
    (9, "Pandey", "HR", 65000, "2022-08-12"),
    (10, "Nivas Krishna", "HR", 45000, "2023-03-22"),
    (11, "Javeed Dutta", "Finance", 70000, "2022-10-05"),
    (12, "Chanikya", "Finance", 55000, "2023-09-15"),
    (13, "Viswak Sen", "Sales", 60000, "2022-11-20"),
    (14, "Charan Sai", "Sales", 50000, "2023-04-14"),
    (15, "Navadeep", "Sales", 40000, "2024-01-10")
]

columns = ["EmployeeID", "EmployeeName", "Department", "Salary", "JoiningDate"]
```

a. **Display the schema and first five records. (1 mark)**

```
# Task 7.a: Print schema structure and display the top 5 rows
df.printSchema()
df.show(5)
```

```
root
 |-- EmployeeID: long (nullable = true)
 |-- EmployeeName: string (nullable = true)
 |-- Department: string (nullable = true)
 |-- Salary: long (nullable = true)
 |-- JoiningDate: string (nullable = true)

+----------+------------------+----------+------+-----------+
|EmployeeID|      EmployeeName|Department|Salary|JoiningDate|
+----------+------------------+----------+------+-----------+
|         1|      Ramesh Kumar|        IT|100000| 2021-01-10|
|         2|      Priya Sharma|        HR| 80000| 2021-03-15|
|         3|        Amit Patel|   Finance| 90000| 2021-05-20|
|         4|       Sneha Reddy|     Sales| 85000| 2021-07-01|
|         5|Yekkaluri Anirudh|        IT| 75000| 2022-02-15|
+----------+------------------+----------+------+-----------+
only showing top 5 rows
```

b. Filter employees whose monthly salary is greater than ₹60,000. (1 mark)

```
[ ]
# Task 7.b: Filter for salary greater than 60000
high_salary_df = df.filter(df.Salary > 60000)
high_salary_df.show()
```

```
+----------+------------------+----------+------+-----------+
|EmployeeID|      EmployeeName|Department|Salary|JoiningDate|
+----------+------------------+----------+------+-----------+
|         1|      Ramesh Kumar|        IT|100000| 2021-01-10|
|         2|      Priya Sharma|        HR| 80000| 2021-03-15|
|         3|        Amit Patel|   Finance| 90000| 2021-05-20|
|         4|       Sneha Reddy|     Sales| 85000| 2021-07-01|
|         5|Yekkaluri Anirudh|        IT| 75000| 2022-02-15|
|         6|     Vikram Rathod|        IT| 75000| 2022-04-10|
|         9|            Pandey|        HR| 65000| 2022-08-12|
|        11|      Javeed Dutta|   Finance| 70000| 2022-10-05|
+----------+------------------+----------+------+-----------+
```

c. Add an AnnualSalary column calculated from the monthly salary. (1 mark)

```
# Task 7.c: Use withColumn to calculate AnnualSalary
df_with_annual = df.withColumn("AnnualSalary", df.Salary * 12)
df_with_annual.show(5)
```

```
+----------+------------------+----------+------+-----------+------------+
|EmployeeID|      EmployeeName|Department|Salary|JoiningDate|AnnualSalary|
+----------+------------------+----------+------+-----------+------------+
|         1|      Ramesh Kumar|        IT|100000| 2021-01-10|     1200000|
|         2|      Priya Sharma|        HR| 80000| 2021-03-15|      960000|
|         3|        Amit Patel|   Finance| 90000| 2021-05-20|     1080000|
|         4|       Sneha Reddy|     Sales| 85000| 2021-07-01|     1020000|
|         5|Yekkaluri Anirudh|        IT| 75000| 2022-02-15|      900000|
+----------+------------------+----------+------+-----------+------------+
only showing top 5 rows
```

d. Group employees by department and calculate the average salary and employee count. (2 marks)

```
# Task 7.d: Group by department and aggregate average salary and count
from pyspark.sql.functions import avg, count

dept_summary = df.groupBy("Department").agg(
    avg("Salary").alias("AverageSalary"),
    count("EmployeeID").alias("EmployeeCount")
)
dept_summary.show()
```

```
+----------+------------------+-------------+
|Department|     AverageSalary|EmployeeCount|
+----------+------------------+-------------+
|     Sales|           58750.0|            4|
|        HR|63333.333333333336|            3|
|   Finance| 71666.66666666667|            3|
|        IT|           72000.0|            5|
+----------+------------------+-------------+
```

e.Display the five highest-paid employees. (1 mark)

```
# Task 7.e: Order by salary in descending order and show top 5
top5_paid = df.orderBy(df.Salary.desc()).limit(5)
top5_paid.show()
```

```
+----------+----------------+----------+------+-----------+
|EmployeeID|    EmployeeName|Department|Salary|JoiningDate|
+----------+----------------+----------+------+-----------+
|         1|     Ramesh Kumar|        IT|100000| 2021-01-10|
|         3|       Amit Patel|   Finance| 90000| 2021-05-20|
|         4|      Sneha Reddy|     Sales| 85000| 2021-07-01|
|         2|     Priya Sharma|        HR| 80000| 2021-03-15|
|         5|Yekkaluri Anirudh|        IT| 75000| 2022-02-15|
+----------+----------------+----------+------+-----------+
```

Data Cleaning and Aggregation

Question 8: Data Cleaning and Aggregation (7 Marks)

```
# a sample transactions.csv file with duplicates, nulls, and bad dates
csv_content = """TransactionID,CustomerID,TransactionDate,Product,Quantity,UnitPrice
1,101,2025-01-10,Laptop,1,50000
2,102,2025-01-15,Mouse,3,500
3,101,2025-02-05,Keyboard,2,1500
1,101,2025-01-10,Laptop,1,50000
4,103,bad-date,Laptop,2,50000
5,104,2025-03-02,Monitor,,15000
6,102,2025-03-12,,5,500
7,105,2025-03-25,Laptop,1,50000
8,103,2025-04-05,Keyboard,2,1500
"""

with open("transactions.csv", "w") as f:
    f.write(csv_content)

print("transactions.csv created successfully!")
```

```
transactions.csv created successfully!
```

a. Read the CSV file using an appropriate schema and header configuration. (1 mark)

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, to_date

spark = SparkSession.builder.appName("DataCleaning").getOrCreate()

# Read CSV with header=True and inferSchema=True
df_raw = spark.read.csv("transactions.csv", header=True, inferSchema=True)
df_raw.show()
```

```
+-------------+----------+---------------+--------+--------+---------+
|TransactionID|CustomerID|TransactionDate| Product|Quantity|UnitPrice|
+-------------+----------+---------------+--------+--------+---------+
|            1|       101|     2025-01-10|  Laptop|       1|    50000|
|            2|       102|     2025-01-15|   Mouse|       3|      500|
|            3|       101|     2025-02-05|Keyboard|       2|     1500|
|            1|       101|     2025-01-10|  Laptop|       1|    50000|
|            4|       103|       bad-date|  Laptop|       2|    50000|
|            5|       104|     2025-03-02| Monitor|    NULL|    15000|
|            6|       102|     2025-03-12|    NULL|       5|      500|
|            7|       105|     2025-03-25|  Laptop|       1|    50000|
|            8|       103|     2025-04-05|Keyboard|       2|     1500|
+-------------+----------+---------------+--------+--------+---------+
```

c. Identify and remove duplicate transactions using a suitable business key

```
# Remove duplicate records based on TransactionID
df_dedup = df_raw.dropDuplicates(["TransactionID"])
df_dedup.show()
```

```
+-------------+----------+---------------+--------+--------+---------+
|TransactionID|CustomerID|TransactionDate| Product|Quantity|UnitPrice|
+-------------+----------+---------------+--------+--------+---------+
|            1|       101|     2025-01-10|  Laptop|       1|    50000|
|            2|       102|     2025-01-15|   Mouse|       3|      500|
|            3|       101|     2025-02-05|Keyboard|       2|     1500|
|            4|       103|       bad-date|  Laptop|       2|    50000|
|            5|       104|     2025-03-02| Monitor|    NULL|    15000|
|            6|       102|     2025-03-12|    NULL|       5|      500|
|            7|       105|     2025-03-25|  Laptop|       1|    50000|
|            8|       103|     2025-04-05|Keyboard|       2|     1500|
+-------------+----------+---------------+--------+--------+---------+
```

c. Handle missing values in required fields and explain your chosen approach. (2 marks)

```
# Drop rows where critical fields (Product or Quantity) are missing/null
df_clean = df_dedup.dropna(subset=["Product", "Quantity", "UnitPrice"])
df_clean.show()
```

```
+-------------+----------+---------------+--------+--------+---------+
|TransactionID|CustomerID|TransactionDate| Product|Quantity|UnitPrice|
+-------------+----------+---------------+--------+--------+---------+
|            1|       101|     2025-01-10|  Laptop|       1|    50000|
|            2|       102|     2025-01-15|   Mouse|       3|      500|
|            3|       101|     2025-02-05|Keyboard|       2|     1500|
|            4|       103|       bad-date|  Laptop|       2|    50000|
|            7|       105|     2025-03-25|  Laptop|       1|    50000|
|            8|       103|     2025-04-05|Keyboard|       2|     1500|
+-------------+----------+---------------+--------+--------+---------+
```

d.Convert TransactionDate into a date type and handle invalid dates. (1 mark)

```
from pyspark.sql.functions import col, to_date

# Keep only rows where TransactionDate looks like a valid date (e.g. 2025-01-10)
df_valid_format = df_clean.filter(col("TransactionDate").rlike("^[0-9]{4}-[0-9]{2}-[0-9]{2}$"))
df_valid_dates = df_valid_format.withColumn("TransactionDate", to_date(col("TransactionDate"), "yyyy-MM-dd"))
df_valid_dates.show()
```

```
+-------------+----------+---------------+--------+--------+---------+
|TransactionID|CustomerID|TransactionDate| Product|Quantity|UnitPrice|
+-------------+----------+---------------+--------+--------+---------+
|            1|       101|     2025-01-10|  Laptop|       1|    50000|
|            2|       102|     2025-01-15|   Mouse|       3|      500|
|            3|       101|     2025-02-05|Keyboard|       2|     1500|
|            7|       105|     2025-03-25|  Laptop|       1|    50000|
|            8|       103|     2025-04-05|Keyboard|       2|     1500|
+-------------+----------+---------------+--------+--------+---------+
```

e.Calculate revenue using Quantity × UnitPrice. (1 mark)

```
# Add Revenue column
df_with_revenue = df_valid_dates.withColumn("Revenue", col("Quantity") * col("UnitPrice"))
df_with_revenue.show()
```

```
+-------------+----------+---------------+--------+--------+---------+-------+
|TransactionID|CustomerID|TransactionDate| Product|Quantity|UnitPrice|Revenue|
+-------------+----------+---------------+--------+--------+---------+-------+
|            1|       101|     2025-01-10|  Laptop|       1|    50000|  50000|
|            2|       102|     2025-01-15|   Mouse|       3|      500|   1500|
|            3|       101|     2025-02-05|Keyboard|       2|     1500|   3000|
|            7|       105|     2025-03-25|  Laptop|       1|    50000|  50000|
|            8|       103|     2025-04-05|Keyboard|       2|     1500|   3000|
+-------------+----------+---------------+--------+--------+---------+-------+
```

f. Calculate total revenue by product and display the results in descending order. (1 mark)

```
from pyspark.sql.functions import sum

# Group by Product, sum Revenue, and sort descending
product_revenue = df_with_revenue.groupBy("Product") \
    .agg(sum("Revenue").alias("TotalRevenue")) \
    .orderBy(col("TotalRevenue").desc())

product_revenue.show()
```

```
+--------+------------+
| Product|TotalRevenue|
+--------+------------+
|  Laptop|      100000|
|Keyboard|        6000|
|   Mouse|        1500|
+--------+------------+
```

Question 9: PySpark Joins and Performance (7 Marks)

a. Load both datasets into DataFrames. (1 mark)

```
customers_data = [
    (1, "Ramesh", "Bengaluru"),
    (2, "Priya", "Mumbai"),
    (3, "Amit", "Delhi"),
    (4, "Sneha", "Hyderabad"),
    (5, "Anirudh", "Bengaluru")  # Anirudh has not placed any orders yet
]
customers_cols = ["CustomerID", "CustomerName", "City"]
df_customers = spark.createDataFrame(customers_data, customers_cols)
orders_data = [
    (101, 1, 50000, "2025-01-10"),
    (102, 2, 1500, "2025-01-15"),
    (103, 1, 25000, "2025-02-05"),
    (104, 3, 60000, "2025-02-20"),
    (105, 4, 3500, "2025-03-02")
]
orders_cols = ["OrderID", "CustomerID", "OrderAmount", "OrderDate"]
df_orders = spark.createDataFrame(orders_data, orders_cols)
print("Customers:")
df_customers.show()
print("Orders:")
df_orders.show()
```

```
df_orders.show()
```

```
Customers:
+----------+------------+---------+
|CustomerID|CustomerName|     City|
+----------+------------+---------+
|         1|      Ramesh|Bengaluru|
|         2|       Priya|   Mumbai|
|         3|        Amit|    Delhi|
|         4|       Sneha|Hyderabad|
|         5|     Anirudh|Bengaluru|
+----------+------------+---------+

Orders:
+-------+----------+-----------+----------+
|OrderID|CustomerID|OrderAmount| OrderDate|
+-------+----------+-----------+----------+
|    101|         1|      50000|2025-01-10|
|    102|         2|       1500|2025-01-15|
|    103|         1|      25000|2025-02-05|
|    104|         3|      60000|2025-02-20|
|    105|         4|       3500|2025-03-02|
+-------+----------+-----------+----------+
```

b. Join the datasets to display orders with customer details. (2 marks)

```
# Task 9.b: Inner Join Orders with Customers on CustomerID
df_joined = df_orders.join(df_customers, on="CustomerID", how="inner")
df_joined.show()
```

```
+----------+-------+-----------+----------+------------+---------+
|CustomerID|OrderID|OrderAmount| OrderDate|CustomerName|     City|
+----------+-------+-----------+----------+------------+---------+
|         1|    101|      50000|2025-01-10|      Ramesh|Bengaluru|
|         1|    103|      25000|2025-02-05|      Ramesh|Bengaluru|
|         2|    102|       1500|2025-01-15|       Priya|   Mumbai|
|         3|    104|      60000|2025-02-20|        Amit|    Delhi|
|         4|    105|       3500|2025-03-02|       Sneha|Hyderabad|
+----------+-------+-----------+----------+------------+---------+
```

c. Identify customers who have never placed an order. (1 mark)

```
# Task 9.c: Left anti join finds customers whose ID does not exist in Orders
customers_no_orders = df_customers.join(df_orders, on="CustomerID", how="left_anti")
customers_no_orders.show()
```

```
+----------+------------+---------+
|CustomerID|CustomerName|     City|
+----------+------------+---------+
|         5|     Anirudh|Bengaluru|
+----------+------------+---------+
```

d. Calculate the total order value by city. (1 mark)

```
from pyspark.sql.functions import sum

# Task 9.d: Group joined data by City and calculate total order amount
city_sales = df_joined.groupBy("City").agg(sum("OrderAmount").alias("TotalCitySales"))
city_sales.show()
```

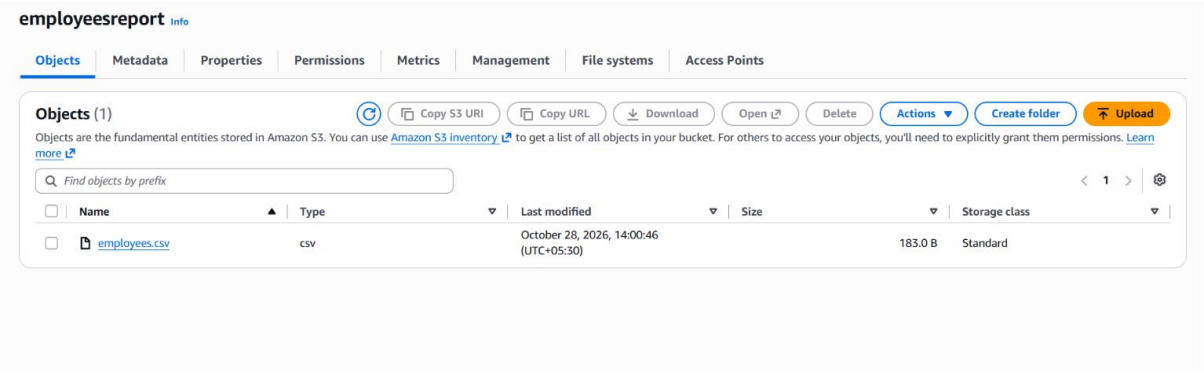
```
+---------+--------------+
|     City|TotalCitySales|
+---------+--------------+
|   Mumbai|          1500|
|    Delhi|         60000|
|Bengaluru|         75000|
|Hyderabad|          3500|
+---------+--------------+
```

SECTION D: AWS CLOUD SERVICES || Maximum Marks: 15

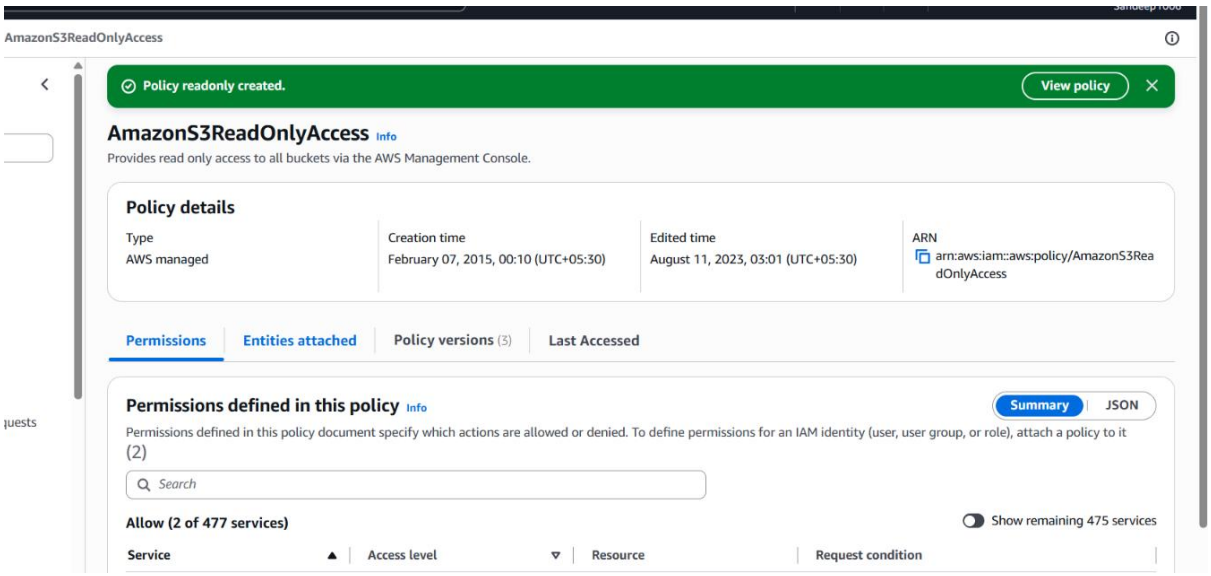
Question 10: S3 and IAM Security (5 Marks)

a. Create an S3 bucket with Block Public Access enabled and upload a sample file.

(1 mark)



b. Design an IAM policy that grants read access to objects in the designated bucket without granting access to unrelated buckets. (2 marks)



c. Explain how you would verify that an unauthorized user cannot read the file. (1 mark)

ANS:

Check permissions: Verify that the file's access permissions allow only authorized users to read it.

Test with an unauthorized user: Log in as a user who does not have permission to access the file.

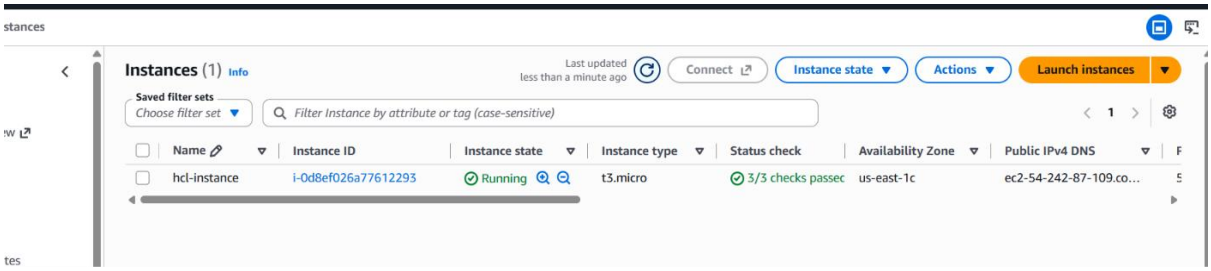
Attempt to read the file: Try opening or downloading the file.

d. Explain how encryption at rest protects the stored data. (1 mark)

ANS: It will not let any unathurised user to read the data it will protect from the inknown users

Question 11: EC2 Troubleshooting and CloudWatch (5 Marks)

a. Check the EC2 instance state and status checks. (1 mark)



b. Explain how security groups and route tables can affect application connectivity. (1 mark)

ANS:

Security groups may block traffic if the required ports are not allowed. Incorrect route table entries can prevent traffic from reaching the EC2 instance.

c. Explain how to configure a CloudWatch alarm when CPU utilization exceeds 85% for a defined evaluation period. (2 marks)

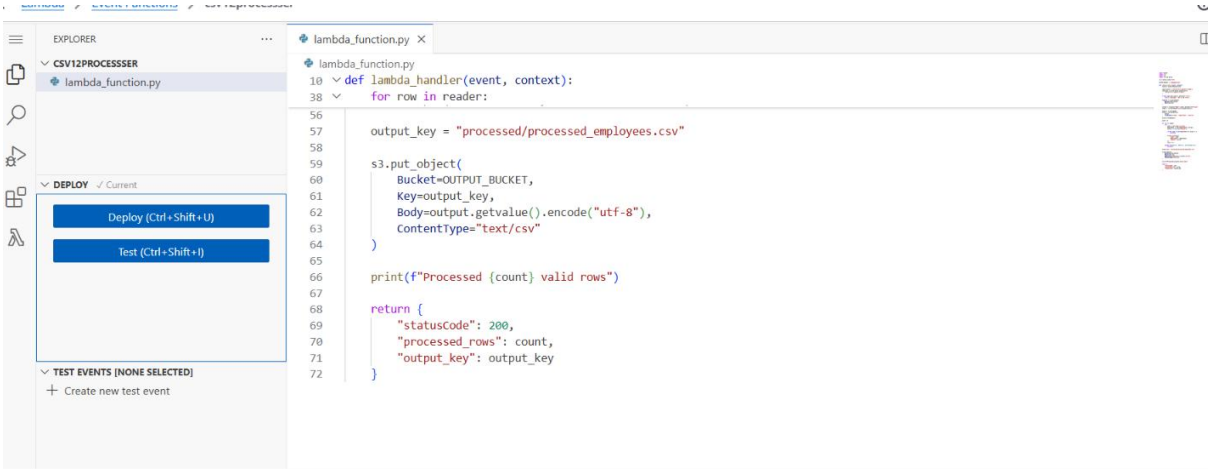
ANS:
Open CloudWatch -Alarms - Create alarm. Select the EC2 CPUUtilization metric, set the threshold to greater than 85%, define the evaluation period, and configure an SNS notification if required. Create the alarm.

d. Identify one method for investigating the root cause of high CPU utilization. (1 mark)

ANS:
Use the CloudWatch CPU utilization graph to identify when CPU usage increases, then check running processes and application logs to find the cause.

Question 12: Event-Driven Processing (5 Marks)

a. Identify a suitable service for executing the processing code. (1 mark)



b. Explain how an S3 upload can trigger processing. (1 mark)

ANS:

An S3 upload triggers processing using S3 Event Notifications, which can invoke a Lambda function or send an event to SQS/EventBridge automatically.

c. Explain the IAM permissions needed to read the input file and write the results. (1 mark)

ANS:

The IAM role needs s3:GetObject permission to read the input file and s3:PutObject permission to write the processed results to the S3 bucket.

d.Explain how errors and processing logs can be captured. (1 mark)

ANS:

Errors and processing logs can be captured using Amazon CloudWatch Logs, which stores Lambda execution logs and helps monitor errors and failures**.**

e.Explain how to prevent duplicate processing or recursive triggering. (1 mark)

ANS:

Use idempotency checks to prevent duplicate processing and store processed file details. Avoid recursive triggering by using separate input and output S3 buckets or different folder prefixes**.**

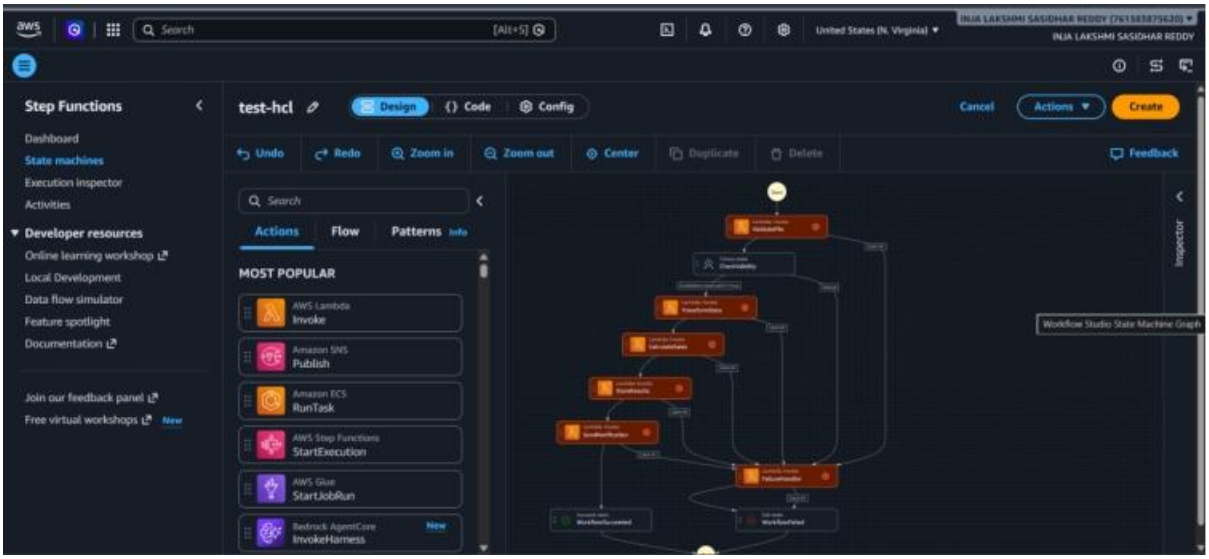
12: Event-Driven Processing (5 Marks)

a. Identify a suitable service for executing the processing code.
Aws Lambda is the best choice. It allows you to run code without provisioning servers, scales automatically, and integrates tightly with S3 events.
b. Explain how an S3 upload can trigger processing.
You configure an S3 Event Notification on the bucket. When a new CSV file is uploaded (e.g., PUT event), S3 sends an event to Lambda, which automatically invokes the function to validate and process the file.
c. Explain the IAM permissions needed to read the input file and write the results.
s3:GetObject permission to read the uploaded CSV file from the input bucket.
s3:PutObject permission to write the processed results to the output bucket or location. Optionally, s3:ListBucket if listing objects is required.
d. Explain how errors and processing logs can be captured.
Amazon CloudWatch Logs automatically captures Lambda execution logs, including errors and custom logging statements.
You can also configure CloudWatch Alarms or DLQs (Dead Letter Queues) with Amazon SQS/SNS to handle failed processing events.

e. Explain how to prevent duplicate processing or recursive triggering.
Use separate input and output buckets (or distinct prefixes) so that writing results does not trigger the same Lambda again.
Add object metadata or DynamoDB tracking to ensure each file is processed only once.
Configure event filters in S3 to trigger only on specific prefixes or suffixes (e.g., .csv).

Question 13: State Machine Design

a. Identify suitable states for validation, processing, storage, and notification
ANS: use Task states to run AWS Lambda functions or AWS SDK integrations for validation, data transformation/calculation, S3 upload, and SNS notification; optionally add a Choice state after validation for branching and a Fail or Succeed state at termination points.



b. Explain how a Choice state can route invalid files to a failure path and valid files to processing.

ANS: Choice State Routing: A Choice state inspects a boolean or status field in the validation output (e.g., $.isValid == true) to route valid files to the processing Task state and invalid files via Default or matching condition directly to a Fail state.

c. Explain how Retry and Catch can handle transient failures and permanent errors.

ANS: Retry vs. Catch: Retry automatically re-executes a failed state with configurable intervals and backoffs to overcome temporary glitches like network timeouts, while Catch intercepts unrecoverable or persistent errors to cleanly divert execution to a fallback or error-handling state.

c. **Explain how the state machine can invoke Lambda functions or other supported AWS services.**

ANS: Invoking Lambda & AWS Services: The state machine invokes services using a Task state by specifying the target service's ARN in the "Resource" field (such as a Lambda ARN or an optimized service integration like arn:aws:states:::sns:publish)

e. Explain how execution history helps troubleshoot a failed workflow.

ANS:Execution History Troubleshooting: The execution history logs a chronological step-by-step audit of state entries, exits, timestamps, exact input/output payloads, and detailed error tracebacks to quickly pinpoint the exact failure point.

Question 14: Step Functions Implementation

a. Write an ASL JSON definition containing the three sequential tasks. Use placeholders for resource ARNs where necessary.
ANS: equential execution: AWS Step Functions executes ValidateFile, TransformData, and LoadResults in sequence using Task states.
b. Configure a retry policy for the transformation task with an appropriate error category, retry interval, and maximum attempts.

ANS: A Choice state checks validation results. If the file is invalid, the workflow stops using a Fail state.

c. Configure a Catch transition for the loading task that routes to an error-handling state.

ANS: Retry: The TransformData task uses Retry with IntervalSeconds: 2, MaxAttempts: 3, and BackoffRate: 2 to handle temporary failures.

d. **Explain how data from one task is passed to the next task.**
ANS: Catch: The LoadResults task uses Catch to route permanent errors to RecordFailure for logging.

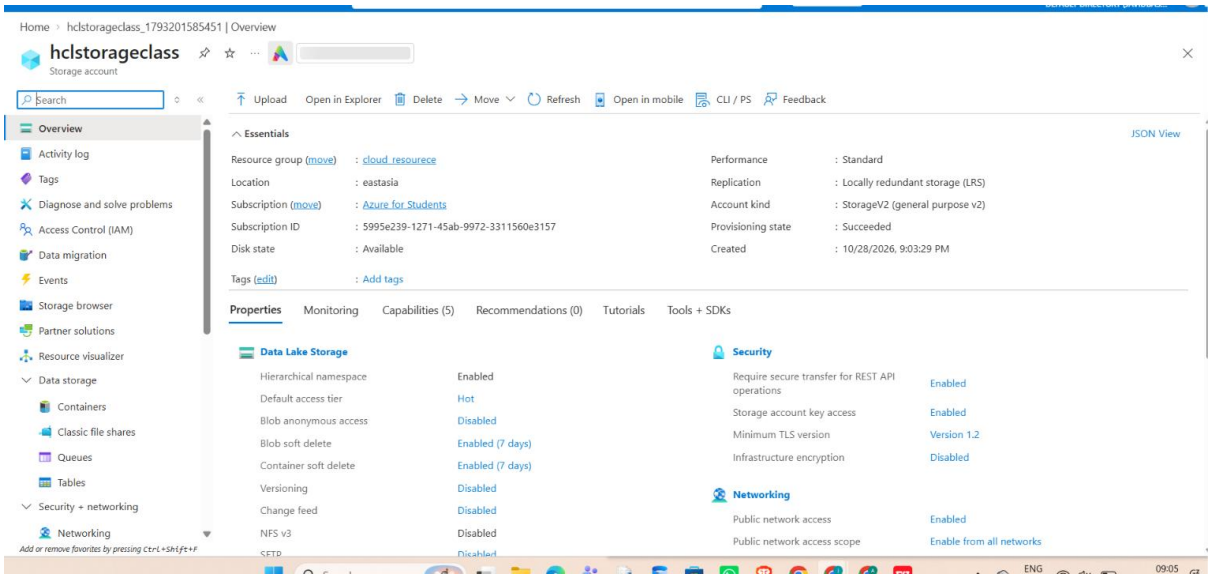
e. **Explain how you would test successful and failed executions.**

ANS: Data passing and testing: Payload.$ passes input between tasks, while ResultPath stores task outputs. Execution history helps debug errors and verify successful or failed runs.

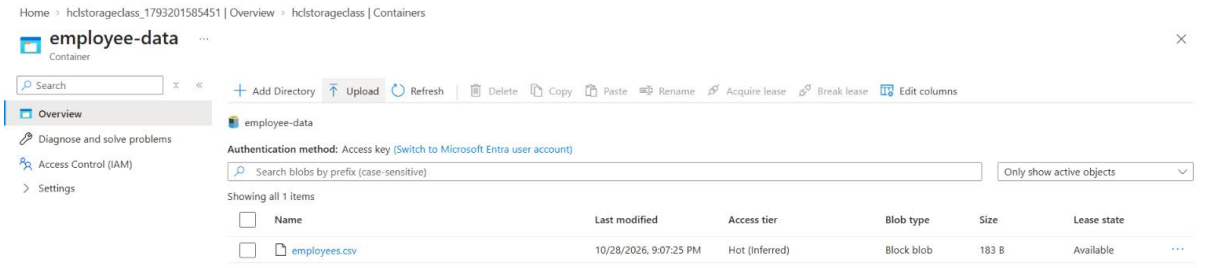
SECTION F: MICROSOFT AZURE || Maximum Marks: 15

Question 15: Azure Data Lake Storage and Security (5 Marks)

a. Configure or describe a storage account with hierarchical namespace enabled for Azure Data Lake Storage Gen2. (1 mark)



b. Create a filesystem or container and upload a sample dataset. (1 mark)



c. Explain how Microsoft Entra ID and Azure role-based access control can manage access to the data. (2 marks)

- Microsoft Entra ID: Authenticates users and verifies their identities.
- Azure RBAC: Assigns permissions to users or groups based on their roles.
- Storage Blob Data Reader: Allows users to read and download data.
- Storage Blob Data Contributor: Allows users to read, upload, and modify data.

(d) How to verify unauthorized users cannot access the dataset (1 mark)

- Assign access only to authorized users.
- Sign in as an unauthorized user and try to access or download the CSV file.
- Verify that access is denied with an authorization error.
- Check Azure Monitor or diagnostic logs, if configured, for access attempts.

Question 16: Azure VM Troubleshooting (5 Marks)

a. Verify the VM's provisioning and power states. (1 mark)

ANS:

Azure VM provisioning state indicates whether the VM was created or configured successfully, while power state indicates whether the VM is running or stopped.

(b) How NSG rules can prevent connectivity (1 mark)

- Check whether inbound traffic allows the required port, such as TCP 22 for Linux SSH or TCP 3389 for Windows RDP.
- Check the source IP, destination port, priority, and any deny rules.
- A higher-priority deny rule can block the connection.

(c) How subnet and routing affect connectivity (1 mark)

Verify that the VM's network interface is connected to the correct subnet.

Check route tables and user-defined routes for incorrect or missing routes

Ensure the required path exists between the administrator's system and the VM.

Check peering, VPN, or gateway configuration if the VM is accessed through a private network.

(d) Secure troubleshooting without exposing the VM publicly (2 marks)

Use Azure Bastion to connect to the VM through the Azure Portal without assigning a public IP to the VM.

Alternatively, use Azure VPN Gateway or a point-to-site VPN to access the VM privately.

Use Network Watcher → IP flow verify to determine whether NSG rules allow or deny traffic.

Question 17: Azure Data Integration (5 Marks)

Question: A company receives a CSV file containing daily customer transactions and needs to prepare it for analysis. Design a solution using Azure services.

a. Select an appropriate service to orchestrate data movement. (1 Mark)

Answer: Azure Data Factory (ADF) is used to orchestrate and automate data movement between different data sources and destinations.

b. Explain how to copy the CSV file into Azure Data Lake Storage Gen2. (1 Mark)

Answer: Use the Copy Data activity in Azure Data Factory to copy the CSV file from the source to Azure Data Lake Storage Gen2.

c. Describe how to organize files by ingestion date. (1 Mark)

Answer: Store files in date-based folders, such as `ingestion_date=2026-10-28`, to organize and track daily data.

d. Explain how to handle malformed records and missing values. (1 Mark)

Answer: Use Azure Databricks with PySpark to validate data, handle missing values, and move malformed records to a separate quarantine folder.

e. Explain how to monitor pipeline execution and validate data completeness. (1 Mark)

Answer: Use Azure Data Factory Monitor to track pipeline status and errors. Compare source and destination row counts to verify data completeness.